Tôn Phương Uyên N25DECE075 lớp E2SCQCE02-N

Problem 1: Given a singly linked list: ~~head → [10]~~

head → [10] → [25] → [40] → [15] → NULL

- Task 1: Write a function / algorithm in pseudocode to reverse the singly ~~linked~~ linked list, including three pointers (prev, curr, next) to complete your function

= To reverse the list, use three pointers: prev, curr, and next

Start with prev = NULL and curr = head. At each step, save the next node, reverse the current pointer, then ~~now~~ move to the next node

```
Reverse (head):
    prev ← NULL
    curr ← head
    next ← NULL
    while curr ≠ NULL:
        next ← curr.next
        curr.next ← prev
        prev ← curr
        curr ← next
    head ← prev
    return head
```

~~⇒ The reversed list is:~~

~~15 → 40 → 25 → 10 → NULL~~

- Task 2: Trace the execution steps (via text description or pointer diagrams) when inserting new value = 5 into the given list

- Task 2:

Draw a diagram or step-by-step pointer trace table showing the positions and values of the three pointers (prev, curr, next) after each iteration until the list is fully reversed

~~1.75~~ 1.00

Step	prev	curr	next
Initial	NULL	10	25
1	10	25	40
2	25	40	15
3	40	15	NULL
4	15	NULL	NULL

Linked Sum ?

~~1.75~~

Final list:

15 → 40 → 25 → 10 → NULL

Problem 2: ~~1.25~~ 2.5

Given a singly circular linked list head → [10] → [20] → [30] → (points back to head)

1.25 • Task 1 : Write pseudocode for the ~~funcition~~ function Inser Head (head, newValue) to insert a new node containing newValue at the front of the circular linked list ~~whill~~ while maintaining its circular structure

~~Ex:~~

Node*

Insert Head (head, value):

    newNode ← Node(value)

0.25 if head = NULL: ~~// head = NULL : List rong~~

        newNode.next ← newNode

        return newNode

0.5   tail ← head

        while (tail.next ≠ head):

            tail ← tail.next

0.5   newNode.next ← head

      tail.next ← newNode

Scanned with CamScanner

head ← newNode

return head

Task 2 Trace the execution steps (via ~~steps~~ text description or pointer diagrams) when inserting newValue = 5 into the given list

Initial list:

10 → 20 → 30 → back to 10

Step 1 Or Create a new node 5

newNode = 5

1,625

Step 2: Find the last node

tail = 30

Step 3 Connect 5 to the old head

5 → 10

Step 4: connect 30 to 5 and update head

5 → 10 → 20 → 30 → back to 5

2,5 Problem 3:

Given a doubly linked list containing integers

9,75 · Task 1 Write pseudo code for InsertBefore (head, target, newValue) to insert a new node containing newValue immediately before the first occurrence of target

```
InsertBefore (head, target, value):
    curr ← head
    while curr ≠ NULL and curr.value ≠ target:
        curr ← curr.next
    if curr = NULL:
        return head
```

Scanned with CamScanner

newNode ← Node (value)

newNode.next ← curr

newNode.prev ← curr.prev

if curr.prev ≠ NULL:

curr.prev.next ← newNode

else:

head ← newNode

curr.prev ← newNode

return head

0.75 Task 2 List the exact order of pointer assignment required and briefly justify why this specific sequence avoids memory leaks or broken links

1. newNode.next ← curr

2 newNode.prev ← curr.prev

3. curr.prev.next ← newNode

4. curr.prev ← newNode

For example, insert 15 before 20:

10 ⇄ 20 ⇄ 30

⇒ After insertion

10 ⇄ 15 ⇄ 20 ⇄ 30

1. Task 3 Explicitly handle the following two edge cases in code / pseudocode:

1. target is located at the head node (head)

2. target does not exist in the list

yet it's the head: set head = newNode

Target does not exist: keep the list unchanged

Problem 4: 2.5

Given a singly linked list sorted in ascending order

head → [2] → [5] → [8] → NULL

1.25 Task 1 Write the function SortedInsert(head, val) to insert val into the list such that the ascending order is preserved

```
SortedInsert(head, value):
    newNode ← Node(value)
    If (head = NULL or value < head.values):
        newNode.next ← head
        head ← newNode
        return head
    curr ← head
    while (curr.next ≠ NULL and curr.next.value < value):
        curr ← curr.next
    newNode.next ← curr.next
    curr.next ← newNode
    return head
```

1.25 Task 2: Construct a step by step trace table detailing pointer iterations and condition evaluations when inserting val = 6 into the list

Initial list:

2 → 5 → 8 → NULL

Scanned with CamScanner

Step 1 curr = 2

5 < 6 → move to 5

Step 2 curr = 5

8 < 6 → False → stop

Step 3 Insert 6 before 8

2 → 5 → 6 → 8 → NULL

Problem 1: 25

Problem 2: 25

Problem 3: 25

Problem 4: 25

100

Scanned with CamScanner